Title of Project Proposal

Yining Yang 89628688
Hazel Dube 9179677
Athulya Shaji 9867600
Mohammed Raiyan Diwan 9967679
Pranjal Anupkumar Satpute 9967643
Chaturthi Ganesh Malvankar 7878503
Vincent James Barila 4491993

October 20, 2026

Abstract

People now send personal, academic, and work messages through web chat, so confidentiality depends on who holds the keys, not only on a platform saying that traffic is protected. WhatsApp already encrypts chats inside its own clients. This project does not replace that protection, and it does not treat WhatsApp as insecure. On WhatsApp Web, a user cannot run a separate encryption step, with keys held only by a browser extension, before the page sends the text. This project proposes a Chrome extension that adds an experimental extra layer to one-to-one text. In the first phase, keys stay in the browser and are not stored on a project server. Each user has an ECDH P-256 key pair. From the popup, the user can turn the extra encryption on or off, see the current status, copy the local public key, paste the other person's public key, and get a warning when a required key is missing. The two browsers derive a shared secret and then an AES-256-GCM key with HKDF and SHA-256, using a new random IV for each message. On send, the extension reads the compose box and replaces the plaintext before WhatsApp transmits it. If encryption fails, the plaintext is not sent. On receipt, the extension finds the payload in the open chat and decrypts it locally. The result is a testable client-side workflow on an existing web messenger, and a clear limit of manual key exchange: a pasted key is not yet evidence of the right person. Key checks, separate keys per contact, rotation, and group chats are left to a later phase.

Statement of Contribution Weights:

Names	Weights
Yining Yang	100%
Hazel Dube	100%
Athulya Shaji	100%
Mohammed Raiyan Diwan	100%
Pranjal Anupkumar Satpute	100%
Chaturthi Ganesh Malvankar	
Vincent James Barila	

Explanations: Chaturthi was limited participation in team meetings and discussions, including meeting absences. She had agreed to review and verify the tasks completed by other team members, but this responsibility was not fulfilled and was ultimately undertaken by Pranjal. Her assigned tasks were delayed, and her research did not produce findings that could be effectively incorporated into the project. Her contribution to the proposal made limited use of independently developed research and was primarily based on materials prepared by other team members.

Vincent joined the team in Week 9 following the lecturer's arrangement and therefore participated in approximately half of the eight-week project period. During this time, he completed a limited part of the development plan within the agreed timeframe. However, his overall contribution remained substantially smaller than that of the members who participated throughout the project. His initial work also required extensive clarification and follow-up communication, which increased the team's coordination workload.

1 Introduction

Digital messaging platforms are extensively utilized to share personal, academic, and professional information, and ensuring the confidentiality and integrity of sensitive messages has become a major security concern. To address this concern, end-to-end encryption (E2EE) is used, which encrypts message content on the sender's device so that only the intended recipient can decrypt it. However, the security of a messaging workflow relies not just on the encryption algorithm, but also on how keys are generated, exchanged, stored, and associated with the correct users.

WhatsApp already provides end-to-end encryption for personal messages and calls. This project does not aim to replace these security methods or suggest that WhatsApp is insecure. Instead, it examines the design of a Chrome browser extension that adds an experimental, extra encryption layer to text messages exchanged through WhatsApp Web. The project explores adding client-side cryptographic processing to an existing web messaging interface by encrypting messages within the browser before sending. The extension is designed as a prototype to study encryption workflows, key handling, and the practical challenges of embedding security functionality into a dynamic website.

The initial phase concentrates on a minimum viable product (MVP). Through a popup interface, users will be able to enable or disable the extension, generate cryptographic key pairs, and manually exchange public keys. The extension will use Elliptic-Curve Diffie–Hellman (ECDH) key agreement to derive a shared secret, which will be used for message encryption and decryption with AES-GCM. When a message is sent, the extension will detect the send action, intercept the plaintext, encrypt it, and replace it with a ciphertext string before transmission through WhatsApp Web. On the receiving end, it will identify encrypted messages, decrypt them, and display the resulting plaintext to the recipient. The popup will also contain key-related controls and show the status of the extension.

A key challenge is that encryption alone does not guarantee that keys have been exchanged with the intended party. Because public keys are exchanged manually in the MVP, the workflow is straightforward to demonstrate but offers no independent authentication of the other participant's identity. If an attacker substitutes a public key during the exchange, users may derive a secret with the wrong party, creating a potential man-in-the-middle risk. Key authentication and management must therefore be considered alongside message encryption. The project must also address practical integration issues, including changes to the WhatsApp Web interface, reliable detection of message events, and clear handling of decryption failures, since the extension must coordinate cryptographic operations with an interface it does not control.

The second phase is to improve the prototype's security, reliability, and usability. Prospective work includes better key authentication, per-contact key management, key rotation, group messaging, secure local key storage, improved message handling, and security and performance testing. These are planned extensions, not features assumed to be in the MVP. Evaluation will check whether the core workflow works as intended and highlight limitations in the implementation. Overall, the project provides a step-by-step approach to implementing and evaluating an extra client-side encryption layer for WhatsApp Web, demonstrating its main workflow, and identifying the limitations that must be addressed before a more comprehensive implementation is considered.

1.1 Aims

The main goal of this project is to create, develop and test a Chrome browser extension that illustrates a new client-side layer of encryption for the text messages sent and received via WhatsApp Web. Content of the message will be handled in the browser but without using a server to store users' cryptographic keys. The project has been split into an initial MVP and a subsequent portion of foreseen security and usability improvements.

The following specific aims are to:

1. **Develop the extension interface and integration.** Include a pop-up to allow users to either enable or disable the extension, as well as access public-key exchange controls and view

the encryption status; and integrate the extension into WhatsApp Web's message-sending and message-receiving workflow.

2. **Implement key establishment.** Create key pairs, import/export public keys manually, and use ECDH key agreement to derive a shared secret between two parties.
3. **Encrypt and decrypt text messages.** Use AES-GCM to encrypt outgoing plaintext before it is sent via WhatsApp Web, and decrypt supported incoming ciphertext to read the original message sent by the sender.
4. **Test the core MVP workflow.** Confirm public-key export and import, shared-secret agreement, message encryption/decryption, and the handling of relevant message actions and errors.
5. **Investigate security limitations and future improvements.** In Phase 2, investigate the problems of public-key substitution and man-in-the-middle attacks in manual public-key exchange, and examine better key authentication and key management, key rotation, group messaging, secure local storage, and better reliability and usability.
6. **Evaluate the prototype.** Evaluate the core workflow to determine if it is working as desired, and determine any security, integration, performance, and usability concerns that would guide future development.

The expected initial outcome is a working prototype demonstrating manual public-key exchange and encrypted text messaging using WhatsApp Web. The features in Phase 2 are treated as planned development and are evaluated according to the project's progress.

1.2 Related Work

WhatsApp End-to-End Encryption and Key Transparency. WhatsApp makes use of the Signal Protocol to lay down the basis of E2EE (end-to-end encrypted) communication. Meanwhile, the Key Transparency feature helps users ensure that their conversation keys have not been covertly replaced [1, 2]. Auditable Key Directory is created to make it possible to verify the correctness of the directory independently. Automation and large-scale deployment decrease the need for manual verification of security codes. However, all of the mentioned features are incorporated into WhatsApp clients themselves and do not support verification of keys controlled by third-party extensions.

Signal Protocol: X3DH and Double Ratchet. The X3DH protocol of the Signal network generates a common key based on authenticated public keys and prekeys published by users, thus allowing communication even if the receiver is not online. The next step in this process is that the Double Ratchet algorithm creates new message keys, updating the encryption state, and providing forward secrecy and post-compromise security [3, 4]. This setup provides better security compared to one session key, although the use of prekeys and ratcheting states is required. On the other hand,

MessageGuard. MessageGuard studies the process of adding content-level encryption to existing web applications using browser-based security overlays. This project provides reusable cryptographic building blocks, key management capabilities, and user interface elements, with evaluations carried out on applications such as Gmail and Facebook Chat [5]. It proves that encryption can be added to an application without having to change the underlying service while at the same time addressing the issue of usability. MessageGuard is, however, more of a generic research tool, and not a WhatsApp extension in particular, and the overlays can be broken with any modifications to the website interface and/or message processing procedure.

EriChain. EriChain is an extension for Chrome/Brave browser used for auditing conversations on WhatsApp Web. This extension makes use of signed hash chain per message, ECDSA signatures, HMAC-based commit keys, acknowledgments, and logs that can be exported to help detect deleted,

reordered, or changed messages [6]. The use of EriChain as an extension for WhatsApp Web serves as an example of how to implement content scripts and background service workers. However, EriChain deals with integrity and audit problems but not confidentiality since its repository states that messages are not encrypted.

Code Verify. Code Verify is an open source browser extension that is designed to detect anomalies in the code sent by WhatsApp Web. Code Verify is able to accomplish this task via comparison of cryptographic hashes of the sent code against a reference code using the verification procedure that employs Cloudflare [7]. It is a tool for detecting any suspicious changes in the app. However, code verification alone does not mean encryption of the message or authentication of the keys controlled by a separate extension.

Messaging Layer Security (MLS). Messaging Layer Security (MLS) refers to an IETF standard of an asynchronous group key agreement protocol that supports groups with few members up to those with thousands of members and provides both forward secrecy and post-compromise security [8]. This tree structure allows for fast key update upon changes to the group membership, and hence, MLS can act as a good point of reference for future group-messaging implementation. MLS requires coordinated group membership management, message format standardization, and use of epochs to track states. It is not safe to share one AES key since it cannot handle any change in membership or compromised keys.

1.2.1 Comparison and Proposed Project Contribution

The reviewed solutions address confidentiality, key verification, message integrity, code integrity, and group key management. Table 1 summarises their strengths, limitations, and differences from the proposed project.

Table 1: Comparison of related messaging-security solutions

Solution	**Main Strength**	**Key Limitation**	**Project Difference**
WhatsApp E2EE	Native encryption and key transparency	Platform-dependent	Independent extension keys
Signal Protocol	Ratcheting and forward secrecy	Complex key and state management	Simplified DH–HKDF–AES-GCM
MessageGuard	Reusable encryption overlays	Generic and UI-dependent	WhatsApp Web-specific implementation
EriChain	Message integrity auditing	No message confidentiality	Encrypts message contents
Code Verify	Detects modified web code	No contact-key verification	Adds encryption and key verification
MLS	Scalable group key management	Requires coordinated group state	Focuses on one-to-one messaging
Proposed Project	**Encryption and contact-key verification**	**Manual setup; no ratchet or group protocol**	**WhatsApp Web encryption prototype**

The suggested MVP would make use of the manual exchange of public keys, out-of-band fingerprint verification, ephemeral Diffie-Hellman, HKDF, and AES-GCM in order to add an independent encryption layer to WhatsApp Web. However, this solution is lacking the sophisticated key management mechanisms of more developed cryptosystems, such as Signal.

It would be desirable to separate cryptographic functions from web-page scripts as much as possible [9]. Even with a compromised browser, plaintext can be revealed. Moreover, both parties will need a proper extension installed. The evaluation should include encryption, decryption error handling, rekeying, UI modifications, and verification functionality.

2 Methodology

This project will implement an end-to-end encryption layer to web-based messaging via a browser-extension. The goal is to keep messages secure until they're received in the recipient's browser where they'll be decrypted. The extension will function without the need to modify the platform itself and will be independent of the messaging service. In the prototype, the selected platform is WhatsApp Web and platform-specific code is retained to allow the approach to be adapted for other web messaging services later on.

The system will be segmented into three sections: extension control module, cryptography module and user interface module. The extension control module will handle the communication between webpage and extension. A content script will be used to track message related actions, and a separate adapter will be used to perform platform specific DOM operations like reading and replacing the content of the message, checking if a conversation is active, and checking for new messages. These operations will be linked with cryptography functions by the background controller. This segregation of the encryption logic from the messaging interface simplifies testing and maintenance and ensures the system's encryption capabilities remain independent of the messaging interface.

The cryptography module will use the Web Crypto API. During Phase 1, each user will have a public and a private key pair, where each key pair is an ECDH key pair. Private key will not be shared or exchanged between the browser extension and other parties. Public keys will be initially exchanged manually via the popup. Each side will then use its own private key and other user's public key to calculate the same shared secret. This common secret will be used to derive an AES-256 key for HKDF with the SHA-256 hash function.

AES-GCM will be used to protect the content of the message as it offers confidentiality and integrity checking. New random IV will be created for each message. The encrypted data will be encased with information needed for decryption such as message version, IV, cipher text and other data. The payload will be Base64 encoded binary in order to be able to be sent as normal text. Decryption should fail if the ciphertext, key, or authentication information is invalid, and should not return invalid plaintext.

The extension will recognize the sending action before the plaintext is sent for outgoing messages. The message will be sent to the background controller for encryption, and the plaintext will be substituted with the encrypted payload before the platform completes the send operation. If the encryption is unsuccessful, the plain text should not be transmitted. Incoming messages will be decrypted and presented locally by the extension if they are in the "encrypted message" format. The user will also be informed about errors, like missing or wrong keys, invalid payloads or failed authentication.

Phase 2 will further improve the one-to-one workflow with enhanced key management capabilities and support for more realistic messaging scenarios. Each contact will have their own key, so that the cryptographic material used in one conversation is not used in another. The encrypted message format shall contain key and version information to be used during decryption to find the right key. Key rotation is scheduled for every 24 hours, and new messages are created with the new key, while old messages are bound to the key version they were created with. Other enhancements that are planned are public-key fingerprints, key-change warnings, replay protection, safer local storage, authenticated key exchange, and encrypted group messaging.

The user interface will be flexible enough that the user does not need to be aware of cryptographic details to follow the security workflow. The popup will let users enable or disable the extension, check if it is encrypted, view and copy the user's public key, and type the other user's public key. The

chat interface will indicate if it is being encrypted or not and will show the decrypted text and helpful error messages. Later versions may also display key-verification status, fingerprints and key-change warnings.

Development and testing will use JavaScript, WebExtensions/Chrome extension APIs, the Web Crypto API, DOM manipulation, Visual Studio Code, Git, and GitHub. Encryption and decryption will be tested, along with IV uniqueness, wrong-key failure, malformed payloads, modified ciphertext, different message lengths, emojis, symbols and multiline text. End to end testing will involve the use of individual browser profiles to simulate individual users and will include tests for sending, receiving, extension restarts and failure handling. Future tests will cover several contacts, key rotation, replaying attempts, group messaging, browser support, and performance. The final evaluation will consider security, usability, reliability, and practical limitations of adding further E2EE in a browser extension.

2.1 Project Challenges

Authenticated Key Exchange One of the main challenges of this project is implementing an authenticated key exchange mechanism. This involves two key difficulties. First, during the initial exchange of public keys, the system must ensure that the received public key genuinely belongs to the intended user rather than to a man-in-the-middle attacker. Without a reliable authentication mechanism, an attacker could intercept the communication and replace the users' public keys with their own. Second, the initial version of the system will use a manual key exchange process, in which Alice sends her public key directly to Bob. Although this approach is relatively straightforward to implement, the ultimate goal is to automate the key exchange process while maintaining the authenticity and security of the exchanged public keys. Designing a secure and user-friendly transition from manual to automatic key exchange will therefore be an important technical challenge.

To address these challenges, the system will use two levels of cryptographic keys: long-term identity keys and conversation-specific ephemeral keys. During the initial authentication process, a human-readable verification code will be generated from the fingerprint of each user's long-term public key. Alice and Bob will compare this code through an out-of-band channel other than WhatsApp. This allows them to verify that the long-term public keys genuinely belong to the intended users. Certificate-based authentication may also be considered as an alternative if a suitable trust model is available.

After the users' long-term identities have been verified, the system will support automatic session-key establishment for individual WhatsApp conversations. When a user opens a conversation, the extension will obtain the conversation ID from the WhatsApp platform. This ID will be used to identify the conversation and to create or retrieve a conversation-specific ephemeral X25519 key pair. The extension will then automatically exchange the ephemeral public keys between the two participants.

After receiving the other participant's ephemeral public key, each party will use X25519 to calculate the same shared secret. HKDF will derive a temporary message-encryption key from this shared secret. The WhatsApp conversation ID, together with the participants' identities and relevant protocol information, will be included in the key-derivation context. As a result, the derived message key will be cryptographically bound to the intended WhatsApp conversation and will not be directly reusable in another conversation.

Group Messaging Encryption Another significant challenge is implementing end-to-end encryption for group conversations. Unlike a one-to-one conversation, a group conversation involves multiple participants, which makes key distribution and management more complex. The system must securely establish a shared group key and ensure that it is available only to authenticated group members.

Group membership may change over time. When a new participant joins, the system must securely distribute the necessary keying material without allowing the new member to decrypt messages sent before they joined. When a participant leaves or is removed, a new group key must be generated and securely distributed to the remaining members so that the former member cannot decrypt fu-

ture messages. Therefore, the system must support secure and efficient key rotation whenever group membership changes.

The group key will be used to derive the temporary message-encryption key for the corresponding WhatsApp group conversation. The WhatsApp conversation ID and relevant protocol information will be incorporated into the key-derivation context to bind the derived key to the intended group. Designing a mechanism that supports authenticated membership, secure group-key distribution, membership changes, and automatic key updates will therefore be an important technical challenge.

Due to its additional complexity, group messaging encryption may be implemented after the one-to-one messaging and key-establishment mechanisms have been successfully developed and evaluated.

2.2 Plan and Time Line

The project will be developed in two phases. Phase 1 will deliver a minimum viable product (MVP) that demonstrates the complete one-to-one encryption workflow. It is expected to require three weeks of development, followed by one week of float for integration, testing, and documentation.

During the first week of Phase 1, the team will establish communication between the extension components, integrate the extension with the WhatsApp Web interface, implement basic key-pair generation, and create the popup interface. The second week will focus on manual public-key exchange, shared-key derivation, outgoing-message interception, AES-GCM encryption, and the initial chat interface. During the third week, the team will implement incoming-message detection, decryption, local key handling, error processing, and interface integration. The fourth week will be reserved for end-to-end testing, defect correction, evaluation, and documentation. The expected outcome of this phase is a functional Chrome extension that demonstrates one-to-one encrypted messaging on WhatsApp Web.

Phase 2 will build on the MVP by improving its security, usability, reliability, and support for multiple conversations and encrypted group messaging. This phase is expected to require eight weeks of development, followed by two weeks of float and finalisation.

Phase 2 will begin with an assessment of the MVP and the design of the conversation and key-management models. The extension will identify the active WhatsApp conversation through its conversation ID and associate each conversation with the correct cryptographic state. Subsequent development will cover automatic public-key exchange, authenticated key establishment, secure local key storage, multi-contact key isolation, versioned encrypted message formats, key rotation, and replay protection.

The prototype will then be extended to support encrypted group conversations and improve the handling of replies, forwarded messages, edited messages, conversation changes, and WhatsApp interface updates. The user interface will display the active contact, encryption status, public-key fingerprint, verification status, and key-change warnings. The final development period will focus on module integration, security testing, performance evaluation, cross-browser compatibility, documentation, and preparation of the final demonstration.

The allocated float periods will be used to address implementation delays, integration problems, and defects discovered during testing. The Phase 1 MVP will be treated as the core deliverable, while the Phase 2 improvements will be implemented according to their priority and technical feasibility.

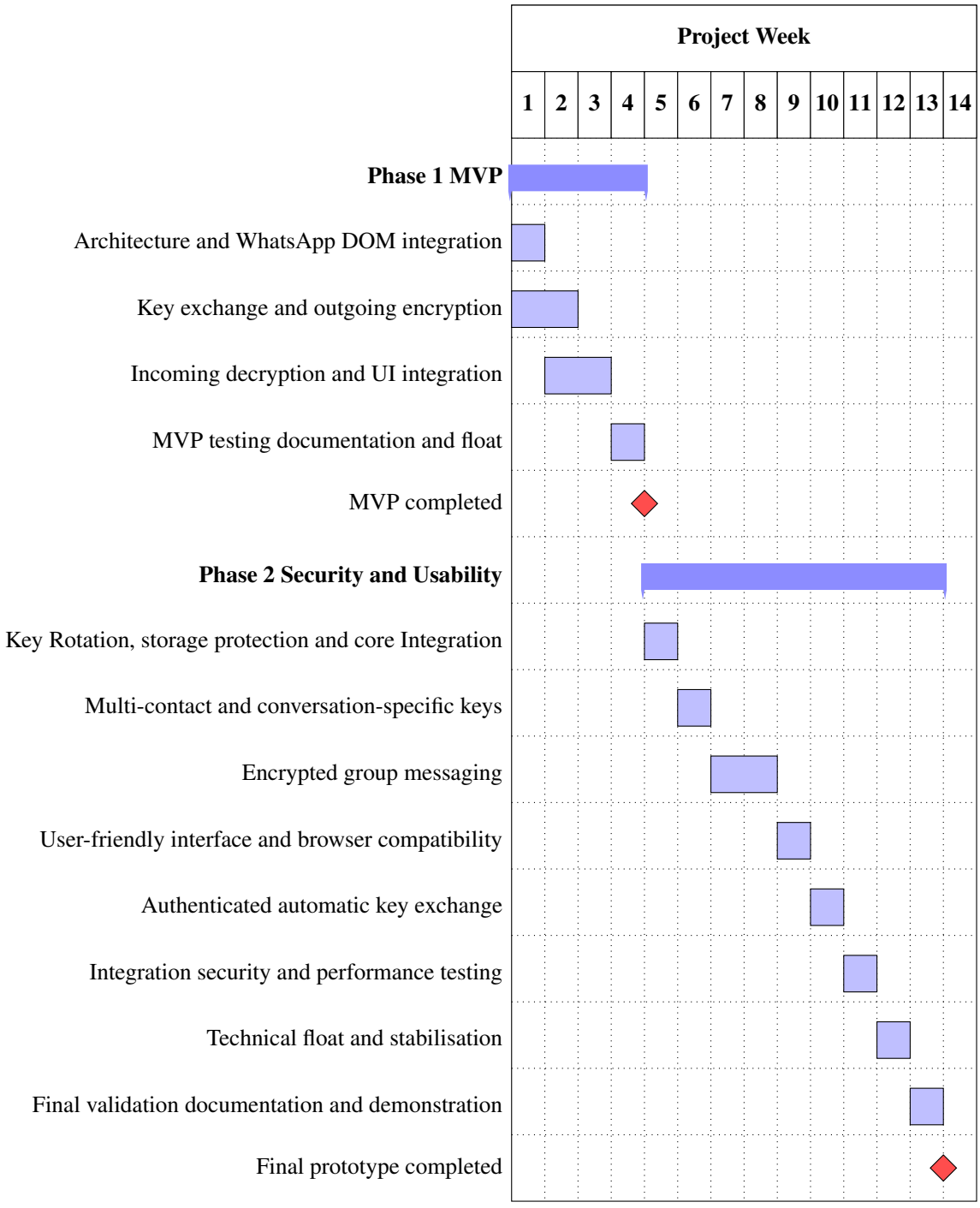


Figure 1: Proposed project development timeline

3 Outcomes

The expected outcome of this project is to develop a functioning browser extension that adds an additional layer of end to end encryption (E2EE) to web based messaging. The extension will be developed for Google Chrome and WhatsApp Web, allowing users to encrypt text messages before they leave the browser and decrypt received messages. This project aims to improve message privacy without needing a separate messaging platform.
The following are the outcomes the project is expected to achieve:

1. **Functional browser extension:** In the first phase, the extension will support sending and receiving encrypted text messages through WhatsApp Web. It will intercept and encrypt outgoing messages before they are sent, detect incoming encrypted messages and display the decrypted content to the recipient.
2. **Key management and message protection:** The extension will generate and store a public and private key pair for each user. Users will exchange public keys manually and use Elliptic Curve Diffie-Hellman (ECDH) to establish a shared secret for deriving encryption keys. AES-GCM will be used to encrypt and decrypt messages. The system will also handle errors such as missing or invalid keys and decryption failures.
3. **User friendly interface:** The popup will allow users to manage encryption settings and show whether encryption is enabled or disabled. Users can also copy their public keys and import a recipient's public key. The interface will display decrypted messages and show relevant error messages when problems occur.
4. **Further improvements:** In the second phase, the team plans to improve the security and usability of the extension. This includes strengthening public key verification to help users confirm that a key belongs to the correct person. The team also plans to introduce key rotation, improve key management to support multiple contacts, and investigate encrypted group messaging. Clearer error messages will help users understand problems when using the extension. These improvements will depend on project progress and feasibility.
5. **Testing and evaluation:** The team will test whether the extension can successfully encrypt and decrypt messages on WhatsApp Web. The evaluation will also consider security aspects such as key management, the usability and performance of the extension, and deployment challenges associated with integrating additional encryption into an existing messaging platform.

The project may benefit users who want more privacy when communicating online, companies protecting confidential information and researchers studying browser security and message privacy. One limitation is that users may not be able to verify whether the public keys they exchange belong to the correct recipients. The extension will also depend on WhatsApp Web, so changes to the platform could affect how the extension works.

4 Conclusion

Overall this project aims to develop an MVP solution on WhatsApp Web and Google Chrome as a web browser extension that adds an additional and experimental layer of client-side encryption to direct messaging. By using ECDH for shared-secret establishment, HKDF with SHA-256 for key derivation, and AES-256-GCM for message encryption, the extension will encrypt before transmission and decrypted locally in the recipient's browser. The project will also provide a simple UI for managing encryption settings without requiring a server to store users cryptographic keys.

The methodology has been discussed which involves developing the extension that adds a client-side encryption layer to WhatsApp Web. The system will consist of an extension control module, a cryptography module, and a user interface, using JavaScript, HTML, CSS, Chrome extension APIs,

and the WebCrypto API. Related works such Signal, MessageGuard, EriChain Code Verify and MLS has been discussed.

The project will be developed in two phases whereas, the first phase will focus on implementing and testing the MVP, including key generation, public-key exchange, message encryption and decryption, and error handling. The second phase will aim to improve the security, reliability, and usability through features such as key verification, key management and key rotation depending on time and feasibility.

Although the proposed extension introduces an additional layer of encryption, it also presents challenges related to authenticated key exchange, browser security, and integration with WhatsApp Web. In particular, manual public-key exchange and group messaging encryption which may affect the extension's functionality. These limitations will be considered during testing and evaluation.

Finally, the project will provide an implementation of browser extensions which introduces additional client-side encryption into an existing messaging platform WhatsApp Web. By evaluating both the functionality and limitations of the proposed approach, the project aims to establish a foundation for future improvements in browser-based message privacy, key management, and secure communication.

References

[1] Meta Engineering, "Making messaging interoperability with third parties safe for users in Europe," Mar. 6, 2024. [Online]. Available: https://engineering.fb.com/2024/03/06/security/whatsapp-messenger-messaging-interoperability-eu/.

[2] Meta Engineering, "Deploying key transparency at WhatsApp," Apr. 13, 2023. [Online]. Available: https://engineering.fb.com/2023/04/13/security/whatsapp-key-transparency/.

[3] Signal, "The X3DH Key Agreement Protocol," Signal Specifications, 2016. [Online]. Available: https://signal.org/docs/specifications/x3dh/.

[4] Signal, "The Double Ratchet Algorithm," Signal Specifications. [Online]. Available: https://signal.org/docs/specifications/doubleratchet/.

[5] S. Ruoti, J. Andersen, T. Monson, D. Zappala, and K. Seamons, "MessageGuard: A Browser-based Platform for Usable, Content-Based Encryption Research," arXiv:1510.08943, 2015. [Online]. Available: https://arxiv.org/abs/1510.08943.

[6] EriChain Extension, "Auditable WhatsApp Web Chats," GitHub repository. [Online]. Available: https://github.com/lucca-poli/eris_extension.

[7] Meta Engineering, "Code Verify: An open source browser extension for verifying code authenticity on the web," Mar. 10, 2022. [Online]. Available: https://engineering.fb.com/2022/03/10/security/code-verify/.

[8] R. Barnes et al., "The Messaging Layer Security (MLS) Protocol," IETF RFC 9420, Jul. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9420.html.

[9] Chrome for Developers, "Stay secure," Chrome Extensions Documentation. [Online]. Available: https://developer.chrome.com/docs/extensions/develop/security-privacy/stay-secure.